Replication Instructions

Paper: Transmission Channel Analysis in Dynamic Models (arXiv:2405.18987)
Repository: https://github.com/enweg/tca-replication-material

Contents

1. Overview
   - Figures in the paper
2. Data availability and provenance
3. Computational requirements
   - Software
   - Hardware, disk, and network
   - Installing the software
4. Obtaining the replication package
   - Main repository
   - MATLAB TCA Toolbox (included)
5. Running the replication
   - Section 5.1: Direct and Indirect Effects of Monetary Policy (Julia)
   - Section 5.2: Anticipation effects of government spending (Julia)
   - Section 5.3: Consumption and Investment Channels of Monetary Policy (MATLAB and Dynare)
6. Output-to-paper mapping
7. Expected warnings (harmless)
8. Repository structure

Overview

These instructions reproduce all results in Section 5 of the paper:

Section	Application	Software	Approx. runtime*
5.1	Direct and Indirect Effects of Monetary Policy (SVAR with internal instruments)	Julia	6 min first run, < 1 min afterwards
5.2	Anticipation effects of government spending (local projections)	Julia	3 min first run, < 1 min afterwards
5.3	Consumption and Investment Channels of Monetary Policy (DSGE model)	MATLAB + Dynare	11 min

*Measured on an Apple M1 (8 cores, 16 GB RAM), macOS 15.7. "First run" includes downloading and precompiling Julia packages. A full replication of all three sections, from a clean clone with no Julia packages installed, took **16 minutes**.

All computations are deterministic (OLS, local projections, and first-order perturbation of a linear DSGE model). No random numbers are drawn, so no seeds are needed. Replication was tested from a clean copy of the repository with Julia 1.13.0 and 1.10.12, and with MATLAB R2024b using Dynare 6.3 and 5.5. All numerical results and figures matched the stored ones, either exactly or up to floating-point rounding (< 1e-11). The only exception is the field `oo_.time` in the `results*.mat` files written by `run00_computation_tca_vs_counterfactual.m`: Dynare stores the computation time of the run there, so this field differs on every run.

Figures in the paper

- **Figure 6** (Section 5.1): `instrument-comparison-GK.pdf` and `instrument-comparison-RR.pdf` together, in `section-5-1/plots/`. Created by `02-plots.jl`.
- **Figure 7** (Section 5.2): `ramey-anticipation-direct-measure.pdf`, in `section-5-2/plots/`. Created by `01-estimation-and-plotting.jl`.
- **Figure 8** (Section 5.3): `tca-in-counterfactual-delta.pdf`, in `section-5-3/outputs/tca-in-counterfactual/`. Created by `run03_plots_tca_in_counterfactual_delta.m`.

The scripts also create two sets of plots that are **not used in the paper**:

- `FRED-OWID-comparison*.pdf`, in `section-5-2/plots/`: diagnostics of the data construction. Created by `00-data-transforms.jl`.
- `sec2-tca-vs-counterfactual.pdf`, in `section-5-3/outputs/tca-vs-counterfactual/`. Created by `run01_plots_section2_tca_vs_counterfactual.m`.

See Output-to-paper mapping for all files the scripts create.

Data availability and provenance

All data needed for replication are included in the repository. No data access applications are required. Section 5.3 uses no data; it only solves a calibrated model.

Data for Section 5.1: `section-5-1/data/mckay_wolf_2023/_data.csv`

A copy of the quarterly dataset from the replication package of McKay, A. and Wolf, C. K. (2023), "What Can Time-Series Regressions Tell Us About Policy Counterfactuals?", *Econometrica* 91(5), https://doi.org/10.3982/ECTA21045. It was obtained on 17/06/2023. `construction.txt` in the same folder documents the original source of every series.

Columns used: `ygap_hp` (output gap), `pgdp` (GDP deflator, converted to annualised inflation), `ffr` (federal funds rate), `lpcom` (log commodity prices), `mp1_tc`

(Gertler–Karadi high-frequency instrument), and `rr_3` (Romer–Romer shocks, Wieland–Yang update). Sample: 1969Q1–2007Q4 (156 quarters). Missing instrument values are set to zero in `01-results.jl`.

Data for Section 5.2: `section-5-2/data/`

Source data in `section-5-2/data/raw/`. `data/raw/README.md` records the retrieval details.

- `RZDAT.xlsx` and `RZDAT.csv`: replication files of Ramey, V. A. and Zubairy, S. (2018), *JPE* 126(2), https://doi.org/10.1086/696277, from https://econweb.ucsd.edu/~vramey/research.html (retrieved 02/11/2023). The CSV is the `rzdata` sheet of the XLSX. Used by `00-data-transforms.jl`.
- `military-expenditure-as-a-share-of-gdp-long.csv`: Our World in Data, https://ourworldindata.org/grapher/military-expenditure-as-a-share-of-gdp-long (retrieved 09/02/2024). Licensed under Creative Commons BY (CC BY 4.0). Recommended citation: Barnum et al. – Global Military Spending Dataset (2025) – with minor processing by Our World in Data. This is the citation Our World in Data currently gives for the series; the file itself was downloaded on 09/02/2024. Used by `00-data-transforms.jl`.
- `Defense.xls`, `Defense_quarterly.csv`, and `Defense_annual.csv`: FRED/ALFRED national defense consumption expenditures and gross investment, series `A997RC1Q027SBEA` (quarterly) and `A997RC1A027NBEA` (annual), vintage 2024-01-25. The CSVs are the two sheets of the XLS. Used by `00-data-transforms.jl`.

Constructed data in `section-5-2/data/`, both created by `00-data-transforms.jl`:

- `spending_quarter.csv`: quarterly defense spending from 1890 onward. OWID annual data are split across quarters using the Ramey–Zubairy government spending shares and spliced with FRED where FRED is available. Intermediate file.
- `data_lp.csv`: the estimation sample, 1890Q1–2015Q1 (501 quarters), with columns `gdef`, `g`, `y`, and `newsy`. Used by `01-estimation-and-plotting.jl`.

Computational requirements

Software

Software	Version tested	Needed for
Julia	**1.13.0** and 1.10.12 (LTS)	Sections 5.1, 5.2
MATLAB	R2024b (Update 5)	Section 5.3
Dynare	**6.3** and 5.5	Section 5.3

- **Julia 1.10 or newer is required.** Julia 1.11.9 and 1.12.7 were also checked: both package environments install on them.
- **MATLAB's Control System Toolbox is required** (`minreal` is called in the TCA toolbox). Parallel Computing Toolbox is optional (a `parfor` loop runs serially without it).
- Dynare is free: https://www.dynare.org/download/.
- The MATLAB TCA Toolbox is included in `tca-matlab-toolbox-v0.3.0/` (see MATLAB TCA Toolbox). Nothing needs to be installed.

The Julia packages are declared in each section's `Project.toml` and are installed automatically on the first run.

- **All packages are pinned to exact versions** in the `[compat]` section of each `Project.toml`. The pinned versions support every Julia version from 1.10 on, so the same package versions are installed on all of them.
  - Section 5.1: CairoMakie 0.15.15, CSV 0.10.17, DataFrames 1.8.2, DataFramesMeta 0.15.6, Distributions 0.25.131, JLD2 0.6.7, TSFrames 0.2.2.
  - Section 5.2: CairoMakie 0.13.10, CSV 0.10.16, Combinatorics 1.0.3, DataFrames 1.8.2, DataFramesMeta 0.15.6, JLD2 0.6.4, StatsBase 0.34.11, TransmissionChannelAnalysis 0.2.1.
- **Julia standard libraries** (`Dates`, `LinearAlgebra`, `Random`, `Statistics`) ship with Julia and cannot be pinned. Their versions follow the Julia version.
- **`TransmissionChannelAnalysis.jl` is installed from its GitHub release v0.2.1**, not from the default branch.
  - On Julia 1.11 and later, this is set in the `[sources]` section of `section-5-2/Project.toml`.
  - Julia 1.10 does not support `[sources]`, so both Section 5.2 scripts install the release explicitly when run on Julia 1.10.
- The Section 5.1 estimation code (VAR, internal instruments, IRFs) is self-contained in `section-5-1/scripts/` and uses only Julia's `LinearAlgebra`. The packages are used only for loading data, saving results, and plotting.

Run `import Pkg; Pkg.status()` from a section folder, after `Pkg.activate(".")`, to list the versions installed on your machine.

Hardware, disk, and network

- Any modern laptop is sufficient, with 8 GB of RAM or more recommended, mainly for precompiling the Julia plotting packages. The tests used an Apple M1 with 16 GB of RAM.
- About 2 GB of disk space for the Julia packages, which are stored in `~/.julia`. The repository itself is 22 MB.

- **An internet connection is required the first time each Julia script runs**, to download packages from the Julia General registry and from GitHub. Section 5.3 needs no internet connection.

Installing the software

- **Julia:** we recommend `juliaup` (https://julialang.org/downloads/). Any release from 1.10 on works. For example, run `juliaup add 1.13` and `juliaup default 1.13`, or use the LTS with `juliaup add lts`. Check with `julia --version`.
- **MATLAB:** follow https://www.mathworks.com/help/install/. Make sure Control System Toolbox is included. Check with `ver control` in MATLAB.
- **Dynare:** install from https://www.dynare.org/download/. Note the path of its `matlab` subfolder. Typical locations are:
  - Windows: `C:/dynare/6.3/matlab`
  - macOS on Apple Silicon: `/Applications/Dynare/6.3-arm64/matlab`
  - macOS on Intel: `/Applications/Dynare/6.3-x86_64/matlab`
  - Linux: `/usr/lib/dynare/matlab`

Obtaining the replication package

Main repository

If you received this package as an archive, for example from the journal's data repository, extract it and use that.

Otherwise, download it from GitHub. **Use the release that matches the paper version you are replicating**, not the default branch. Each paper version is tagged as a release: see the repository's *Releases* page. The default branch may still hold an older version of the code. With git:

```
git clone https://github.com/enweg/tca-replication-material.git
cd tca-replication-material
git checkout <release-tag>
```

Without git: on the *Releases* page, download the release's source code ZIP and extract it.

Below, **`<root>`** denotes the folder that contains `README.md`, `replication-instructions.md`, and the three `section-5-*` folders.

MATLAB TCA Toolbox (included)

`<root>/tca-matlab-toolbox-v0.3.0/` contains an unmodified copy of the MATLAB TCA Toolbox, release v0.3.0 (tag `v0.3.0`, commit `e0416bf`), including

its license. The Section 5.3 scripts add it to the MATLAB path themselves. No action is needed.

Running the replication

> **Important: every script uses paths relative to its own section folder.** Always run a script from inside that folder: `section-5-1`, `section-5-2`, or `section-5-3`. In VS Code, open the section folder itself (File > Open Folder), not `<root>`. In MATLAB, set the current folder to `section-5-3`.
>
> **Warning: running the scripts overwrites the results stored in the repository.** The repository ships with the output of every script. Running a script replaces these files:
>
> - `section-5-1/output/` and `section-5-1/plots/`
> - `section-5-2/plots/`, and also `section-5-2/data/*.csv` if you run `00-data-transforms.jl`
> - `section-5-3/outputs/` and the Dynare-generated files in `section-5-3/dynare-models/`
>
> To keep the original versions, for example to compare them with your own results, copy the whole repository folder before running anything. In a git clone you can also restore them afterwards with `git checkout -- .`. This also discards any other local changes, such as the edited Dynare path.

The three sections are independent and can be run in any order.

Ways to run a Julia script. Each is equivalent to the terminal commands below.

- **Terminal:** use the commands shown in each subsection.
- **Julia REPL:** start `julia`, then run `cd("<root>/section-5-1")` and `include("01-results.jl")`.
- **VS Code** with the *Julia* extension: use **File > Open Folder** to open the *section folder*, open the script, and run **Julia: Execute File in REPL** (the play button at the top right).

The first time each Julia environment is used, the script downloads and precompiles its packages. It then writes a `Manifest.toml` into the section folder; this file is not tracked by git.

Direct and Indirect Effects of Monetary Policy (Julia)

Reproduces Section 5.1 of the paper.

From a terminal:

```
cd <root>/section-5-1
julia 01-results.jl
julia 02-plots.jl
```

- `01-results.jl`: Installs packages on the first run (about 5 min). Estimates a VAR(4) with constant and linear trend, 1969Q1–2007Q4. Identifies the monetary policy shock with internal instruments: Gertler–Karadi (`mp1_tc`) and Romer–Romer (`rr_3`). Decomposes the IRFs into the effect through the contemporaneous FFR response and the remaining effect. The shock is normalised to 25 bp.
  Output:
  `output/gk.jld2`,
  `output/rr.jld2`.
- `02-plots.jl`: Plots the FFR and inflation panels.
  Output (together **Figure 6** in the paper):
  `plots/instrument-comparison-GK.pdf`,
  `plots/instrument-comparison-RR.pdf`.

Anticipation effects of government spending (Julia)

Reproduces Section 5.2 of the paper.

```
cd <root>/section-5-2
julia 00-data-transforms.jl        # optional: rebuilds data/ from data/raw/
julia 01-estimation-and-plotting.jl
```

- `00-data-transforms.jl` (*optional*): Builds the quarterly defense spending series and the estimation dataset from `data/raw/`. Also writes diagnostic plots comparing the OWID-based series with FRED.
  Output:
  `data/spending_quarter.csv`,
  `data/data_lp.csv`,
  `plots/FRED-OWID-comparison*.pdf` (not used in the paper).
- `01-estimation-and-plotting.jl`: Estimates LPs with 4 lags, horizons 0–20, and the Ramey–Zubairy news shock ordered first. Decomposes the responses of GDP and defense spending into an anticipation channel (not through defense spending) and an implementation channel.
  Output (**Figure 7** in the paper):
  `plots/ramey-anticipation-direct-measure.pdf`.

Whichever script runs first installs this section's packages (about 2–3 min). The versions differ from those in Section 5.1, so they are installed separately. On Julia 1.10, both scripts first install `TransmissionChannelAnalysis.jl` from its GitHub release v0.2.1 (see Computational requirements).

Consumption and Investment Channels of Monetary Policy (MATLAB and Dynare)

Reproduces Section 5.3 of the paper.

Step 1: set the Dynare path. Line 3 of each of these two scripts

- `run00_computation_tca_vs_counterfactual.m`
- `run02_computation_tca_in_counterfactual_delta.m`

reads:

```
addpath("/Applications/Dynare/6.3-arm64/matlab/")  % change this to ...
```

Replace the path with the `matlab` subfolder of your Dynare installation (see Installing the software). If Dynare is already on your MATLAB path, leave the line as is. MATLAB then only prints a "Name is nonexistent" warning.

Step 2: run the four scripts in order, with `section-5-3` as the current folder. In the MATLAB GUI, open each file and click **Run**, or type its name in the Command Window.

From a terminal, use the commands below. `matlab` must be on your `PATH`; otherwise use the full path, for example `/Applications/MATLAB_R2024b.app/bin/matlab` on macOS or `"C:\Program Files\MATLAB\R2024b\bin\matlab.exe"` on Windows.

```
cd <root>/section-5-3
matlab -batch "run00_computation_tca_vs_counterfactual"
matlab -batch "run01_plots_section2_tca_vs_counterfactual"
matlab -batch "run02_computation_tca_in_counterfactual_delta"
matlab -batch "run03_plots_tca_in_counterfactual_delta"
```

- `run00_...` (runtime about 40 s): Solves the baseline model (`dynare-models/model_extensive_gap.mod`) and two counterfactual models: $\sigma \to \infty$, i.e. no consumption response (`..._no_consumption.mod`), and $\phi_I \to \infty$, i.e. no investment response (`..._no_investment.mod`). Computes total effects and TCA consumption and investment channels of a monetary policy shock, horizons 0–12.
  Output (in `outputs/tca-vs-counterfactual/`):
  `results*.mat`.
- `run01_...` (runtime about 30 s): Plots the inflation response in three panels (Counterfactual, TCA, TCA closest to counterfactual), horizons 0–8.
  Output (in `outputs/tca-vs-counterfactual/`; not used in the paper):
  `sec2-tca-vs-counterfactual.pdf`.
- `run02_...` (runtime about 9 min): Re-solves the baseline model for depreciation rates $\delta = 0.001, 0.002, \ldots, 0.050$ (50 values) and computes the channels for each.
  Output (in `outputs/tca-in-counterfactual/`):

`resultsDelta-from0p001-by0p001-to0p05.mat`.

- `run03_...` (runtime about 20 s): Plots the responses and channels against δ for $\delta \in [0.0125, 0.05]$.
  Output (in `outputs/tca-in-counterfactual/`; **Figure 8** in the paper): `tca-in-counterfactual-delta.pdf`.

`run01` needs only the output of `run00`, and `run03` needs only the output of `run02`. Neither uses Dynare. Because the `.mat` files are included, the plotting scripts can be run on their own.

Running Dynare regenerates the files in `dynare-models/` (the `+model_*` folders and `model_*/`). These files are tracked in git, so `git status` lists them as modified. This is expected.

Output-to-paper mapping

- Section 5.1, in `section-5-1/plots/`, together **Figure 6**:
  - Gertler–Karadi instrument: `instrument-comparison-GK.pdf`
  - Romer–Romer instrument: `instrument-comparison-RR.pdf`
- Section 5.2, in `section-5-2/plots/`:
  - **Figure 7**, anticipation vs. implementation channels (GDP, defense spending): `ramey-anticipation-direct-measure.pdf`
  - data construction diagnostics (not used in the paper): `FRED-OWID-comparison*.pdf`
- Section 5.3, in `section-5-3/outputs/`:
  - TCA vs. counterfactual (inflation response; not used in the paper): `tca-vs-counterfactual/sec2-tca-vs-counterfactual.pdf`
  - **Figure 8**, channels as a function of the depreciation rate δ: `tca-in-counterfactual/tca-in-counterfactual-delta.pdf`

Expected warnings (harmless)

Where	Message	Why it is harmless
Section 5.1, `02-plots.jl`	`Found `resolution` in the theme ... deprecated` (Makie)	Deprecated keyword; the figure is unaffected.
Section 5.3, Dynare preprocessing	`WARNING: in the 'steady_state_model' block, variable '...' is not assigned a value` (33 times per model)	The model is linearised by hand, so all variables have a zero steady state.

Where	Message	Why it is harmless
Section 5.3, baseline model	`Some of the parameters have no value (rk_bar, s_c, s_I) when using steady`	These parameters are computed by the `steady_state_model` block when `steady` runs.
Section 5.3, no-consumption counterfactual	`Matrix is close to singular or badly scaled`	Intended: $\sigma = 10^{10}$ approximates $\sigma \to \infty$.
Section 5.3, `matlab -batch` only	`CodeCache is full. Compiler has been disabled.`	Java message from MATLAB's graphics stack in batch mode.

Repository structure

```
<root>
|-- README.md
|-- replication-instructions.md        this file
|-- tca-matlab-toolbox-v0.3.0/         MATLAB TCA Toolbox v0.3.0 (included)
|-- section-5-1/
|   |-- Project.toml                   Julia environment
|   |-- 01-results.jl, 02-plots.jl     main scripts
|   |-- scripts/                       helper functions (VAR, IV, plots)
|   |-- data/mckay_wolf_2023/          data (+ construction.txt)
|   |-- output/                        numerical results (.jld2)
|   `-- plots/                         figures
|-- section-5-2/
|   |-- Project.toml
|   |-- 00-data-transforms.jl, 01-estimation-and-plotting.jl
|   |-- scripts/                       helper functions
|   |-- data/raw/                      source data (+ README.md with sources)
|   |-- data/                          constructed data
|   `-- plots/
`-- section-5-3/
    |-- run00_...m - run03_...m            main scripts
    |-- functions/                     TCA channel definitions
    |-- dynare-models/                 .mod files (+ Dynare output)
    `-- outputs/                       results (.mat) and figures
```